

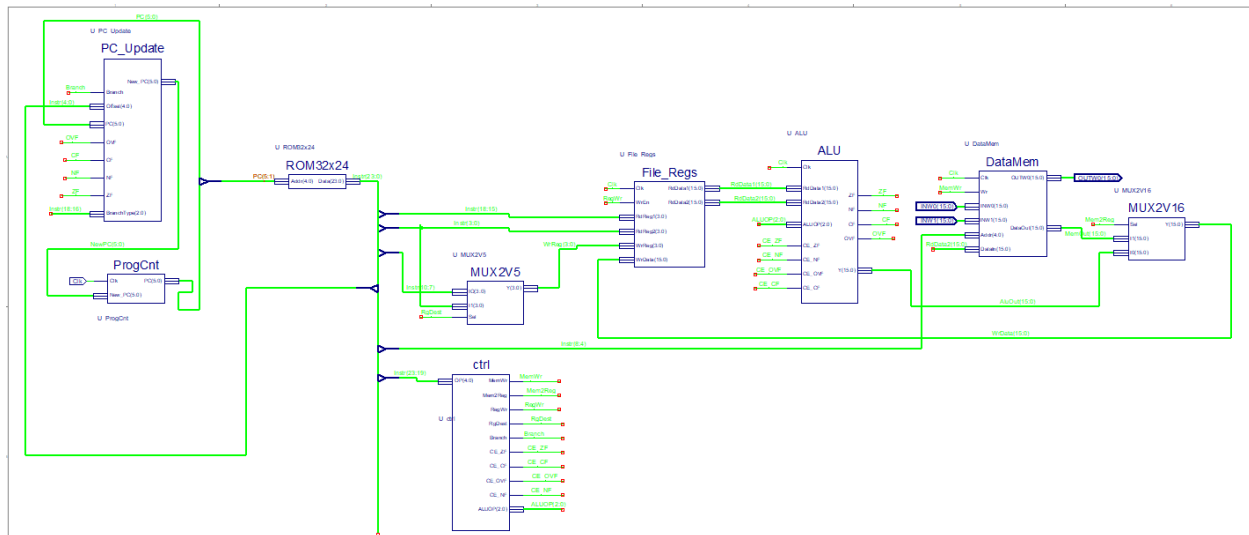
Documentație Proiect PIC24

Număr Proiect: 60

Proiectul 60

Non-jump Instructions	Flags	Jump Instructions
FF1L Ws,Wnd	OV	BRA OV,Expr
SETM Wd	C	BRA C,Expr
SUBB Wb,Ws,Wd	N	BRA N,Expr
RRC Ws,Wd	Z	BRA Z,Expr

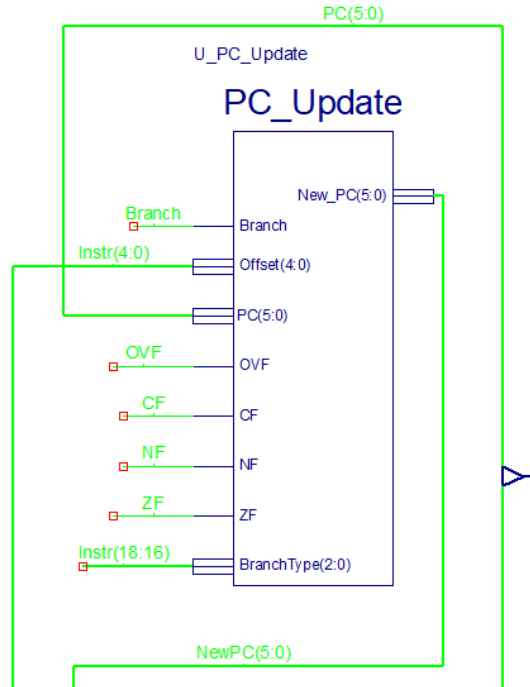
Schema bloc:



Descriere Blocuri

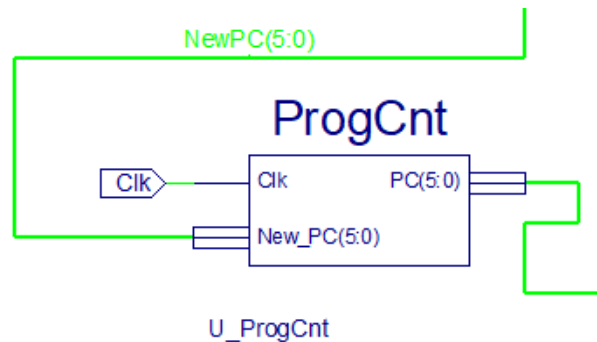
PC_Update:

Este blocul destinat calculului noii valori a PC-ului (program counter). Pentru instrucțiunile non-jump acesta se incrementează din 2 în 2. În cazul unei instrucțiuni jump (salt), program counter-ul se incrementează implicit cu 2 la care se adaugă un deplasament (offset / Instr (4:0)). În funcție de branchType (Instr (18:16)) și valoarea flag-urilor, acesta selectează o anumită instrucțiune de salt condiționată



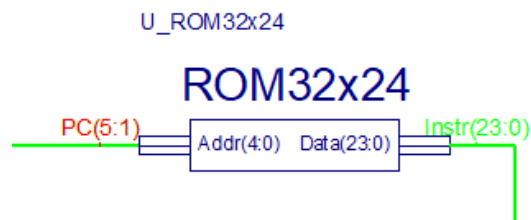
ProgCnt:

Acesta este blocul care actualizează PC cu valoarea calculată în blocul PC_Update, operația de atribuire a noii valori este sincronizată cu un semnal de ceas (Clk)



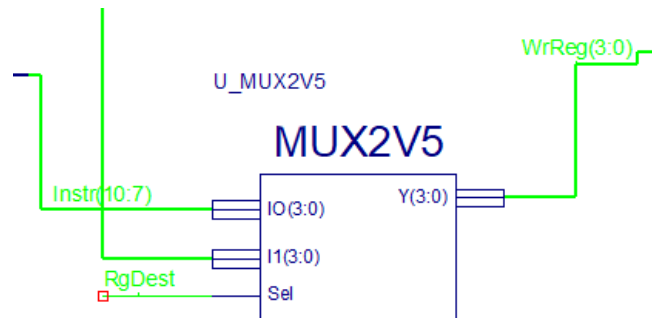
Rom32x24:

Este blocul de memorie a microprocesorului de tip read only, care conține 32 de locații cu 24 de biți fiecare, aici se atribuie codul mașinii generat pentru fiecare instrucțiune.



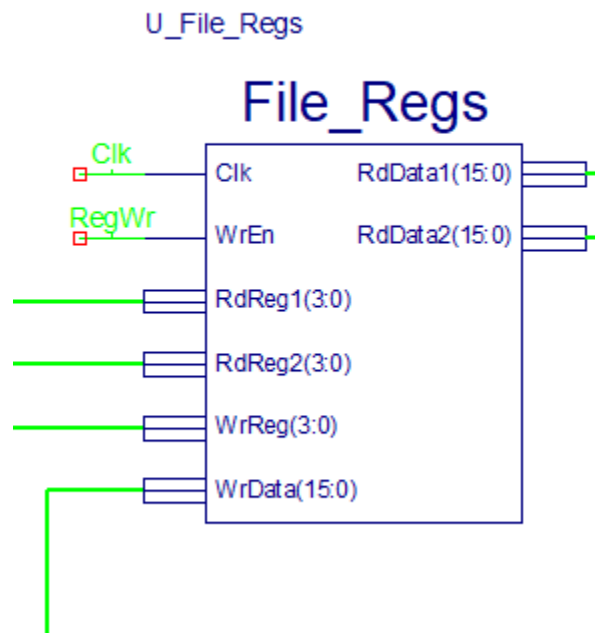
Mux2v5:

Este un mux destinat pentru selecția registrului destinație, acesta fiind pe biții 10:7 sau 3:0 din instrucțiune. Selecția este făcută de către semnalul RgDest care este controlat de blocul ctrl. Astfel când pentru o instrucțiune registrul de destinație este pe biții 3:0, RgDest este 1 și 0 pentru biții 10:7. Ieșirea reprezintă adresa registrului.



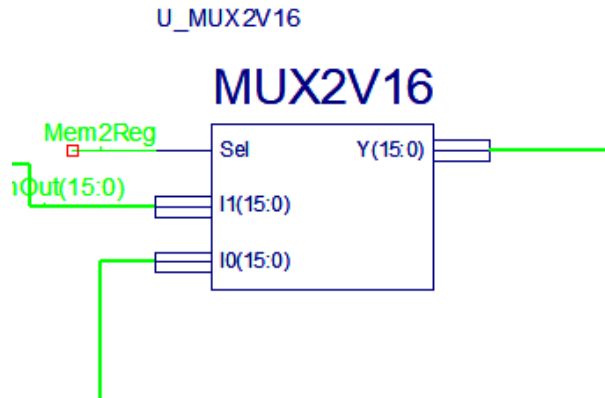
File_regs:

Acest bloc conține registrele folosite pentru realizarea operațiilor. Acesta citește informația din registre pentru registrul baza și sursă și scrie datele pentru registrul destinație.



Mux2v16

Acest mux este folosit pentru a decide sursa de date pentru scrierea în registre, astfel pentru I0 sursa este din ALU și pentru I1 sursa este din memorie, cum ar fii cazul instrucțiunii **MOV f, Wnd**.



ALU:

Acesta este blocul responsabil pentru toate operațiile aritmetice și logice efectuate.

Prin semnalul ALUOP, furnizat de către blocul ctrl, acesta identifică operația pe care trebuie să o efectueze și să furnizeze rezultatul și flag-urile afectate în urma executării. RdData1 (registru baza) și RdData2 (registru sursa) reprezintă operandii necesari pentru execuția instrucțiunii.

Semnalele CE_ZF, CE_NF, CE_OVF, CE_CF reprezintă niște semnale de activare, acestea semnalează că ALU trebuie să furnizeze valoarea flag-urilor care sunt : ZF(zero) ,NF (negativ) ,OVF(overflow) ,CF(carry).

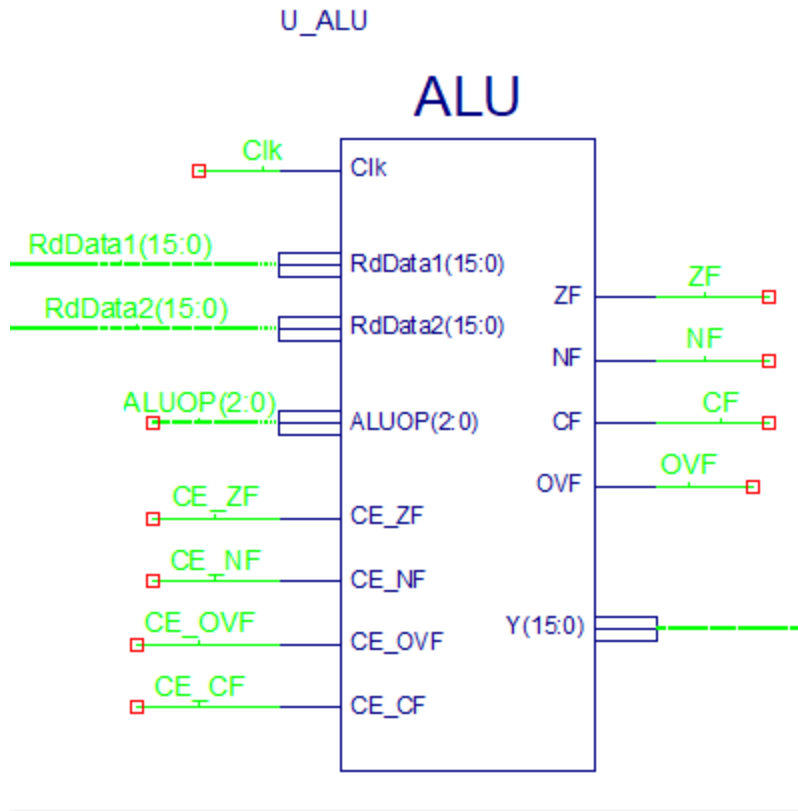
Instrucțiunile comune sunt instrucțiuni care se pot realiza doar cu ajutorul operațiilor “+”, “-”, “and” și “OR”.

Instrucțiunea **FFIL Ws, Wnd** – găsește prima apariția unui “1” pornind de la cel mai semnificativ bit (16) al registrului sursă spre cel mai puțin semnificativ bit (1). Dacă nu a fost găsit nici un bit de “1” atunci flag-ul C va fi setat (1), iar în cazul în care s-a găsit un bit de “1” atunci flag-ul C este șters (0). Aceasta instrucțiune afectează doar flag-ul C.

Instrucțiunea **SETM Wd** - setează toți biții din registrul destinație pe “1”. Aceasta instrucțiune nu afectează nici un flag.

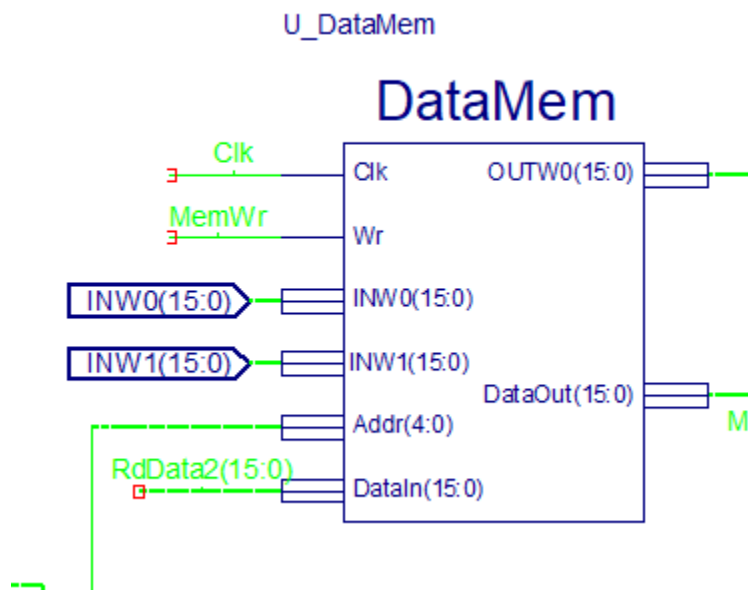
Instrucțiunea **SUBB Wb, Ws, Wd** - aceasta instrucțiune realizează operația de scădere din registrul de bază, scăzând registrul sursă și inversul flag-ului C (not C). Rezultatul fiind pus în registrul destinație. Aceasta instrucțiune afectează flag-urile N, OV, Z, C.

Instrucțiunea **RRC Ws, Wd** – rotește conținutul registrului sursă cu un bit spre dreapta prin flag-ul C și plasează rezultatul în registrul destinație astfel, bitul flag-ului C este plasat pe poziția cea mai semnificativă a registrului destinație și după este subscris cu bitul cel mai puțin semnificativ al registrului sursă. Aceasta instrucțiune afectează flag-urile N, Z, C.



DataMem

Acest bloc reprezintă o memorie cu 16x16 module RAM. Instrucțiunea **MOV Wns, F** scrie datele pentru intrările INW0 și INW1



Ctrl

Este blocul care controlează toate celelalte blocuri astfel.

Primește OPCODE-ul (Instr (23:19)) și în funcție de instrucțiune transmite semnale de control și decizie.

MemWr – este semnalul utilizat în blocul DataMem și activează scrierea în memorie.

Mem2Reg – reprezintă selecția pentru blocul MUX2V16, face selecția pentru sursa de date care se va scrie în registrii

RgDest – reprezintă selecția pentru blocul MUX2V5, astfel se selectează biții unde se afla registrul de destinație

Branch – reprezintă un semnal folosit în blocul PC_Update pentru a semnaliza că avem o instrucțiune de salt

CE_ZF, CE_CF, CE_OVF, CE_NF sunt semnale de activare pentru flag-urile din blocul ALU

ALUOP este semnalul transmis către ALU în funcție de OPCODE pentru a identifica instrucțiunile.

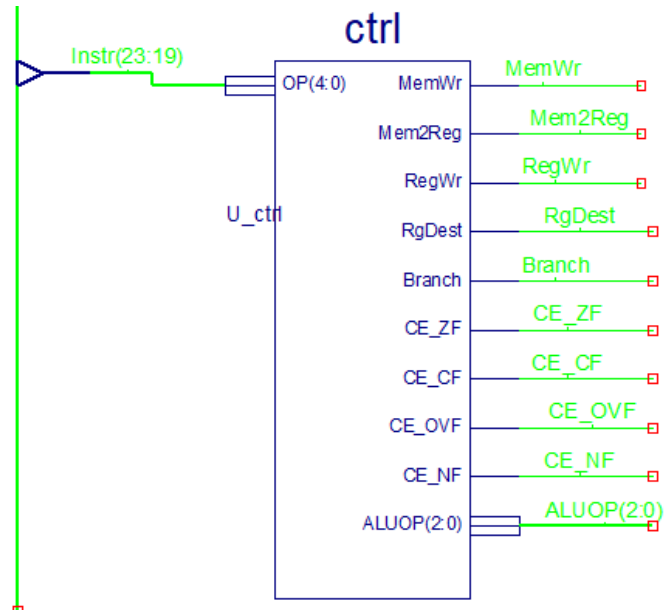


Table Encoding

Encoding	2222 3210	1111 9876	1111 5432	11 1098	7654	3210	Flags
ADD Wb,Ws,Wd	0100	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
SUB Wb,Ws,Wd	0101	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
AND Wb,Ws,Wd	0110	0www	wBqq	qddd	dppp	ssss	N, -, Z, -
IOR Wb, Ws, Wd	0111	0www	wBqq	qddd	dppp	ssss	N, Z
MOV f, <u>wnd</u>	1000	0fff	ffff	ffff	ffff	dddd	None
MOV <u>wns</u> , f	1000	1fff	ffff	ffff	ffff	ssss	None
BRA expr	0011	0111	nnnn	nnnn	nnnn	nnnn	None
FF1L Ws,Wnd	1100	1111	1000	0ddd	dppp	ssss	C
SETM Wd	1110	1011	1Bqq	qddd	d000	0000	None
SUBB Wb,Ws,Wd	0101	1www	wBqq	qddd	dppp	ssss	N, OV, Z, C
RRC Ws,Wd	1101	0011	1Bqq	qddd	dppp	ssss	N, Z, C
BRA OV,Expr	0011	0000	nnnn	nnnn	nnnn	nnnn	None
BRA C,Expr	0011	0001	nnnn	nnnn	nnnn	nnnn	None
BRA N,Expr	0011	0011	nnnn	nnnn	nnnn	nnnn	None
BRA Z,Expr	0011	0010	nnnn	nnnn	nnnn	nnnn	None

Flags

Encoding	CE_NF(CE Flag N)	CE_OVR(CE Flag OV)	CE_ZF(Flag Z)	CE_CF(Flag C)	Flags
ADD Wb,Ws,Wd	1	1	1	1	N, OV, Z, C
SUB Wb,Ws,Wd	1	1	1	1	N, OV, Z, C
AND Wb,Ws,Wd	1	0	1	0	N, -, Z, -
IOR Wb,Ws,Wd	1	0	1	0	N, Z
MOV f, <u>wnd</u>	0	0	0	0	None
MOV <u>wns</u> , f	0	0	0	0	None
BRA expr	0	0	0	0	None
FF1L Ws,Wnd	0	0	0	1	C
SETM Wd	0	0	0	0	None
SUBB Wb,Ws,Wd	1	1	1	1	N, OV, Z, C
RRC Ws,Wd	1	0	1	1	N, Z, C
BRA OV,Expr	0	0	0	0	None
BRA C,Expr	0	0	0	0	None
BRA N,Expr	0	0	0	0	None
BRA Z,Expr	0	0	0	0	None

Ctrl semnale

Table	ing	OPCODE	CE_NF	CE_OVF	CE_ZF	CE_CF	ALUOP	MemWr	Mem2Reg	RegWr	RgDest	Branch
	ADD Wb,Ws,Wd	01000	1	1	1	1	000	0	0	1	0	0
	SUB Wb,Ws,Wd	01010	1	1	1	1	001	0	0	1	0	0
	AND Wb,Ws,Wd	01100	1	0	1	0	010	0	0	1	0	0
	IOR Wb,Ws,Wd	01110	1	0	1	0	011	0	0	1	0	0
	MOV f, wnd	10000	0	0	0	0	x	0	1	1	1	0
	MOV wns, f	10001	0	0	0	0	x	1	0	0	0	0
	BRA expr	00110	0	0	0	0	x	0	x	0	x	1
	FF1L Ws,Wnd	11001	0	0	0	1	100	0	0	1	0	0
	SETM Wd	11101	0	0	0	0	101	0	0	1	0	0
	SUBB Wb,Ws,Wd	01011	1	1	1	1	110	0	0	1	0	0
	RRC Ws,Wd	11010	1	0	1	1	111	0	0	1	0	0
	BRA OV,Expr	00110	0	0	0	0	x	0	x	0	x	1
	BRA C,Expr	00110	0	0	0	0	x	0	x	0	x	1
	BRA N,Expr	00110	0	0	0	0	x	0	x	0	x	1
	BRA Z,Expr	00110	0	0	0	0	x	0	x	0	x	1

Notă

Mem2Reg = 0, sursa de date alu; Mem2reg = 1, sursa de date memoria

RegWr = scrie sau nu in registru (1 scrie, 0 nu scrie)

MemWr = scrie sau nu in memorie (1 scrie, 0 nu scrie)

RegDest = 0 (biti 10-7) RegDest = 1 (biti 3-0)